

Laboratório Prático de Classificação

Implementação do KNN e Curva de Validação em Python

Prof. João Vítor Rodrigues de Vasconcelos

Engenharia de Computação / Análise e Desenvolvimento de Sistemas
joaovasconcelos@univicsa.com.br

ADS308

O que faremos na prática hoje?

- Sair da teoria geométrica e colocar a "mão na massa" com código Python.
- Importar e manipular um dataset real da área da saúde usando pandas.
- Aplicar o pré-processamento obrigatório (StandardScaler).
- Construir uma Curva de Validação para encontrar o K ótimo.
- Treinar o modelo KNN final usando a biblioteca `scikit-learn` e medir sua acurácia.

O Dataset de Hoje: Breast Cancer Wisconsin

- Trabalharemos com um dos datasets mais clássicos de Machine Learning.
- **Objetivo do Negócio (Saúde):** Prever se um tumor mamário é Maligno ou Benigno com base em imagens digitalizadas de biópsias.

Características dos Dados

- **Amostras (Linhas):** 569 pacientes.
- **Features (Colunas):** 30 atributos numéricos (ex: raio médio, textura, perímetro, área, suavidade da célula).
- **Target (Alvo):** 0 para Maligno, 1 para Benigno.

Passo 1: Importando as Bibliotecas

Configuração do Ambiente

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Bibliotecas do Scikit-Learn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

- Execute esta célula no seu Jupyter Notebook para garantir que todas as dependências estão instaladas.

Passo 2: Carregando os Dados no Pandas

Do Scikit-Learn para o DataFrame

```
# 1. Carregar o objeto bruto
```

```
dataset = load_breast_cancer()
```

```
# 2. Criar o DataFrame com as 30 features
```

```
df = pd.DataFrame(data=dataset.data,  
                  columns=dataset.feature_names)
```

```
# 3. Adicionar a coluna alvo (Target)
```

```
df["target"] = dataset.target
```

```
# Visualizar as 5 primeiras linhas
```

```
display(df.head())
```

Por que usar o Pandas?

- O Scikit-Learn armazena os dados em arrays puros do NumPy (formato de dicionário chamado *Bunch*).
- Transformar em um DataFrame do pandas nos permite:
 - Visualizar os dados em formato tabular (linhas e colunas).
 - Verificar rapidamente os nomes das features.
 - Facilitar a integração com bibliotecas de plotagem como Seaborn na fase de Análise Exploratória (EDA).

Passo 3: Separando a Matriz X e o Vetor y

- Antes de dividirmos em Treino e Teste, precisamos separar o que é "pergunta"(features) do que é "resposta"(target).

Definindo as Variáveis

```
# X recebe todas as 30 colunas de características
```

```
X = dataset.data
```

```
# y recebe apenas o diagnóstico (0 ou 1)
```

```
y = dataset.target
```

Passo 4: Divisão de Treino e Teste

Evitando o "Decoreba" do Modelo

```
# Reservando 30% dos dados para testar o modelo depois
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=70,
    stratify=y # Mantém a proporção de tumores
)
```

- **Atenção:** O parâmetro `stratify=y` é crucial na área da saúde para garantir que tenhamos a mesma porcentagem de casos malignos e benignos no treino e no teste.

O Problema das Escalas Diferentes

- O KNN usa a **Distância Euclidiana** para encontrar os vizinhos.
- No nosso dataset, a coluna "Área" pode ter valores na casa dos 1000, enquanto a "Suavidade" tem valores na casa de 0.01.
- Se não fizermos nada, o algoritmo achará que a "Área" é a única variável que importa, pois seus números são absolutos maiores.

Passo 5: Escalonamento (Feature Scaling)

Padronização com StandardScaler

```
# Instanciando o escalonador
scaler = StandardScaler()

# Ajustamos (fit) e transformamos apenas os dados de treino!
X_treino_escalonado = scaler.fit_transform(X_treino)

# Para o teste, usamos apenas transform (usando a média
# e o desvio padrão aprendidos no treino)
X_teste_escalonado = scaler.transform(X_teste)
```

A Busca pelo K Ótimo

- Lembrem-se da aula teórica: como sabemos qual valor de K usar?
- K muito pequeno = **Overfitting** (Memorização de ruído).
- K muito grande = **Underfitting** (Generalização excessiva).

A Estratégia

Vamos criar um *loop* (laço de repetição) que treina um KNN diferente para cada valor ímpar de K (de 1 a 49), calcula o erro e guarda em uma lista para plotarmos a **Curva de Validação**.

Passo 6: Construindo a Curva de Validação

```
taxas_de_erro = []
valores_k = range(1, 50, 2) # K ímpares (1, 3, 5... 49)
for k in valores_k:
    # IDENTAR DESSE COMENTÁRIO
    # FIQUEM ATENTOS COM O FOR, QUANDO COPIAR DO SLIDE E COLAR NO COLAB
    # 1. Cria o modelo com o K atual
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    # 2. Treina o modelo
    knn_temp.fit(X_treino_escalonado, y_treino)
    # 3. Faz a previsão no conjunto de teste
    previsoes = knn_temp.predict(X_teste_escalonado)
    # 4. Calcula o erro (média de acertos invertida)
    erro = np.mean(previsoes != y_teste)
    taxas_de_erro.append(erro)
# ATÉ ESSE
```

Passo 7: Encontrando o Mínimo Global

Extraindo o melhor K via código

```
# Pega o índice numérico onde ocorreu o menor erro
indice_melhor_k = np.argmin(taxas_de_erro)

# Busca qual era o K correspondente a esse índice
k_otimo = valores_k[indice_melhor_k]
menor_erro = taxas_de_erro[indice_melhor_k]

print(f"O K ótimo encontrado foi: {k_otimo}")
print(f"Com taxa de erro de: {menor_erro:.4f}")
```

- Isso evita que a gente tenha que olhar o gráfico e "chutar" qual é o ponto mais baixo do vale.

Passo 8: Plotando a Curva (Visualização)

Código para o Gráfico

```
plt.figure(figsize=(10, 5))
plt.plot(valores_k, taxas_de_erro, color="blue",
         linestyle="dashed", marker="o",
         markerfacecolor="red", markersize=8)

plt.axvline(x=k_otimo, color="green", linestyle="--")
plt.title("Curva de Validação do KNN")
plt.xlabel("Valor de K")
plt.ylabel("Taxa de Erro Média")
plt.grid(True)
plt.show()
```

Interpretando a Curva de Validação

- Ao executar o código no notebook, vocês notarão um formato de "U" ou um "vale".
- A linha verde tracejada marcará exatamente o fundo do vale.
- **À esquerda da linha:** A zona de Overfitting (o modelo está muito sensível).
- **À direita da linha:** A zona de Underfitting (o modelo está ficando brando demais e perdendo a capacidade de distinguir tumores malignos de benignos).

Passo 9: Treinamento Definitivo

Criando o Modelo Final

```
# Agora que sabemos o melhor K, criamos o modelo oficial
modelo_final = KNeighborsClassifier(n_neighbors=k_otimo)

# Treinamos com a base de treino escalonada
modelo_final.fit(X_treino_escalonado, y_treino)

# Geramos as previsões oficiais para a base de teste
previsoes_finais = modelo_final.predict(X_teste_escalonado)
```


Passo 10: Avaliação de Desempenho

Medindo a Acurácia

```
# Comparamos as previsões do modelo com o gabarito real
acuracia = accuracy_score(y_teste, previsoes_finais)

print(f"Acurácia Final do Modelo: {acuracia * 100:.2f}%")
```

- A **Acurácia** nos diz a porcentagem total de acertos (quantos tumores o nosso modelo classificou corretamente do total de pacientes de teste).
- O KNN, mesmo sendo simples, costuma passar de 95% de acurácia neste dataset.

Formato de Entrega

A entrega oficial será um **Relatório Escrito (Word ou PDF)** contendo a análise técnica e de negócio. No cabeçalho do documento, incluam o **link público do Google Colab** com o código da prática. No Portal do Aluno até às 23:59 do dia 08/04/2026

- **1. Impacto do Escalonamento:** Explique por que o uso do StandardScaler foi imprescindível neste dataset. O que aconteceria com o cálculo da Distância Euclidiana se ele não fosse aplicado?
- **2. Interpretação da Curva:** Analise a sua Curva de Validação. Descreva o comportamento do erro quando $K = 1$ e quando $K \geq 35$. Relacione esses extremos aos conceitos de *Overfitting* e *Underfitting*.

- **3. Estratificação dos Dados:** Qual é a função exata do parâmetro `stratify=y` no momento da divisão dos dados? Por que ignorá-lo em um contexto médico pode invalidar o seu modelo?
- **4. Viabilidade Clínica:** Considerando a acurácia final obtida (e a simplicidade do método de distâncias), o algoritmo KNN seria confiável, por si só, para dar o diagnóstico final a um paciente real? Justifique apontando as limitações do modelo.